

SESSION 8

User Authentication System

Topics Covered: User model, Signup, Login, Logout, Password hashing, Authentication backend

Activities: Build Register -> Login -> Logout, Display logged-in username in navbar

Task 1: Create a Django project called musiczone and set up default authentication system for user register, login, logout.

Explanation & Approach: *Django provides built-in authentication models and views under django.contrib.auth.*

```
# musiczone/urls.py
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('accounts/', include('django.contrib.auth.urls')), # Built-in login/logout endpoints
]
```

Task 2: Build a signup form for a food delivery app (like Zomato) collecting username, email, password, and saving users securely with hashed passwords.

Explanation & Approach: *User.objects.create_user handles hashing raw passwords with PBKDF2 algorithm before saving to database.*

```
# accounts/views.py
from django.shortcuts import render, redirect
from django.contrib.auth.models import User
from django.contrib import messages

def signup_view(request):
    if request.method == 'POST':
        username = request.POST['username']
        email = request.POST['email']
        password = request.POST['password']

        if User.objects.filter(username=username).exists():
            messages.error(request, "Username already taken!")
            return redirect('signup')

        # Securely hash and create user
        user = User.objects.create_user(username=username, email=email, password=password)
        messages.success(request, "Account created successfully! Please login.")
        return redirect('login')

    return render(request, 'registration/signup.html')
```

Task 3: Implement a login feature for a movie booking site (BookMyShow style) using Django's authentication backend, redirecting to welcome page.

Explanation & Approach: *authenticate()* verifies credentials against database hashes; *login()* creates session cookies.

```
# accounts/views.py (Custom Login View)
from django.shortcuts import render, redirect
from django.contrib.auth import authenticate, login

def login_view(request):
    if request.method == 'POST':
        u = request.POST['username']
        p = request.POST['password']
        user = authenticate(request, username=u, password=p)
        if user is not None:
            login(request, user)
            return redirect('welcome_page')
        else:
            return render(request, 'registration/login.html', {'error': 'Invalid username
or password!'})
    return render(request, 'registration/login.html')
```

Task 4: Display logged-in user's username in navbar of e-commerce app (Flipkart-style) template using template tags.

Explanation & Approach: *{% if user.is_authenticated %}* conditionally inspects session state inside templates.

```
<!-- templates/navbar.html -->
<nav class="navbar navbar-expand-lg navbar-dark bg-primary">
  <div class="container">
    <a class="navbar-brand font-weight-bold" href="#">Flipkart Clone</a>
    <div class="navbar-nav ms-auto">
      {% if user.is_authenticated %}
        <span class="nav-item nav-link text-white me-3">Welcome, <strong>{{
user.username }}</strong></span>
        <a class="btn btn-outline-light btn-sm" href="{% url 'logout'
%}">Logout</a>
      {% else %}
        <a class="nav-link" href="{% url 'login' %}">Login</a>
        <a class="nav-link" href="{% url 'signup' %}">Signup</a>
      {% endif %}
    </div>
  </div>
</nav>
```

Task 5: Add a logout button to app navbar that logs out user and redirects them to homepage.

Explanation & Approach: *logout(request)* flushes user session keys, destroying active authentication credentials.

```
# accounts/views.py
from django.contrib.auth import logout
from django.shortcuts import redirect

def user_logout(request):
    logout(request)
    return redirect('homepage')
```